

Student 31 **SHAIK SHOUIB AKTHAR** [192312208RB](#)

4 / 20 submitted

Q1. Arrays

CODE

```
#include <stdio.h>

int main() {
    int production[12];
    int month;
    int annual = 0;
    int processedMonths = 0;

    /* Target details */
    int target = 5000;

    /* Input 12 months */
    for (month = 0; month < 12; month++) {
        scanf("%d", &production[month]);

        /* (b) Skip invalid production months */
        if (production[month] == -1 || production[month] == -2) {
            /* If -2 is shutdown, we break; if -1 is invalid, we continue */
            if (production[month] == -2) {
                /* plant shuts down due to equipment failure */
                break;
            } else {
                /* invalid production → skip month */
                continue;
            }
        }

        /* If production is a valid positive number, accumulate */
        if (production[month] ≥ 0) {
            annual += production[month];
            processedMonths++;
        }
    }

    /* To avoid division by zero if all months were skipped/broken immediately */
    if (processedMonths == 0) {
        printf("Annual Production = 0\n");
        printf("Average Monthly Production = 0.00\n");
        printf("Production Efficiency = 0.00%\n");
        printf("Incentive Status = No Incentive\n");
        printf("Inspection Status = Marked for Inspection\n");
        return 0;
    }

    /* (a) Annual, average, efficiency */
    double average = (double)annual / processedMonths;
    double efficiency = (average / target) * 100.0;

    printf("Annual Production = %d\n", annual);
    printf("Average Monthly Production = %.2f\n", average);
    printf("Production Efficiency = %.2f%%\n", efficiency);

    /* (a) Incentive decision (10% if production exceeds target) */
    if (average > target) {
        printf("Incentive Status = Performance Incentive (10%)\n");
    } else {
        printf("Incentive Status = No Incentive\n");
    }

    /* (a) Inspection decision (if below 80% of target) */
    if (average < (0.8 * target)) {
```

```
        printf("Inspection Status = Marked for Inspection\n");
    } else {
        printf("Inspection Status = Not Marked for Inspection\n");
    }

    return 0;
}
```

INPUT

```
5200
4800
5500
3900
5100
-2
5300
5600
4700
5000
-1
5400
```

OUTPUT

(no output)

Q2. Arrays

CODE

```
#include <stdio.h>

int main() {
    int status[20];
    int quality[20];
    int delay[20];

    int excellent = 0, good = 0, average = 0, needs = 0;
    int countActive = 0;

    int i;
    for (i = 0; i < 20; i++) {
        // Read status: 1 active, 0 cancelled, -1 stop
        if (scanf("%d", &status[i]) != 1) return 0;

        if (status[i] == -1) {
            break; // management stops evaluation
        }

        // If cancelled, ignore using continue
        if (status[i] == 0) {
            // Still read quality and delay values if they are provided in input
            // (If input does NOT provide them for cancelled, remove these reads.)
            scanf("%d", &quality[i]);
            scanf("%d", &delay[i]);
            continue;
        }

        // Active project: read quality and delay
        scanf("%d", &quality[i]);
        scanf("%d", &delay[i]);

        countActive++;

        // Classification based on given criteria
        if (quality[i] == 90 && delay[i] == 2) {
            excellent++;
        } else if (quality[i] ≥ 75 && quality[i] ≤ 89 && delay[i] == 5) {
            good++;
        } else if (quality[i] ≥ 60 && quality[i] ≤ 74) {
            average++;
        } else { // quality below 60
            needs++;
        }
    }

    // Compute average quality score and delivery percentage
    // "Delivery percentage" interpreted as percentage of projects meeting delay requirement (Excellent/Good/others)
    // If you meant something else, tell me and I'll adjust.
    int sumQuality = 0;
    int deliveredCount = 0; // projects that satisfy delay conditions for Excellent or Good

    for (int j = 0; j < i; j++) {
        if (status[j] == 0) continue; // cancelled ignored
        sumQuality += quality[j];

        if ((quality[j] == 90 && delay[j] == 2) ||
            (quality[j] ≥ 75 && quality[j] ≤ 89 && delay[j] == 5)) {
            deliveredCount++;
        }
    }
}
```

```
double avgQuality = 0.0;
double deliveryPercent = 0.0;

if (countActive > 0) {
    avgQuality = (double)sumQuality / countActive;
    deliveryPercent = ((double)deliveredCount / countActive) * 100.0;
}

// Output
printf("Excellent: %d\n", excellent);
printf("Good: %d\n", good);
printf("Average: %d\n", average);
printf("Needs Improvement: %d\n", needs);

printf("Average Quality Score: %.2lf\n", avgQuality);
printf("Delivery Percentage: %.2lf%%\n", deliveryPercent);

return 0;
}
```

INPUT

```
1
92
2
1
80
5
1
65
4
0
50
1
-1
```

OUTPUT

(no output)

Q3. Arrays

— Not Submitted —

Q4. Arrays

— Not Submitted —

Q5. Arrays

— Not Submitted —

Q6. Arrays

CODE

```
#include <stdio.h>

int main() {
    int dayCount = 30;
    int chipsPerDay = 10000;

    // Arrays to store status and defective chips for 30 days
    int status[30];
    int defective[30];
    int i;

    // ——— Input for Array Processing (30 days) ———
    // status meaning (as per sample):
    // 0 = maintenance day (skip processing using continue)
    // 1 = machine breakdown (stop using break)
    // 2 = normal production day (compute quality)
    printf("Enter input for 30 days:\n");
    printf("For each day enter: status (0/1/2) and defective chips (only meaningful when status=2)\n");

    for (i = 0; i < dayCount; i++) {
        printf("Day %d status: ", i + 1);
        scanf("%d", &status[i]);

        if (status[i] == 2) {
            printf("Day %d defective chips: ", i + 1);
            scanf("%d", &defective[i]);
        } else {
            defective[i] = 0; // not used for maintenance/breakdown
        }
    }

    // ——— (b) Array processing with continue and break ———
    int dayAcceptedTotal = 0; // total accepted chips across processed days
    int dayDefectTotal = 0; // total defective chips across processed days

    for (i = 0; i < dayCount; i++) {
        if (status[i] == 0) {
            // Maintenance day: skip
            continue;
        }

        if (status[i] == 1) {
            // Machine breakdown: stop processing
            break;
        }

        // ——— (a) Arithmetic operators + decision making for each normal day ———
        // defect percentage = (defective / total) * 100
        float defectPercent = (defective[i] * 100.0f) / chipsPerDay;

        int acceptedChips = chipsPerDay - defective[i];

        // Decide grade based on defect rate
        char grade;
        if (defectPercent < 1.0f) {
            grade = 'A';
        } else if (defectPercent ≥ 1.0f && defectPercent ≤ 3.0f) {
            grade = 'B';
        } else if (defectPercent > 3.0f && defectPercent ≤ 5.0f) {
            grade = 'C';
        } else {
```

```
        grade = 'R'; // Reject
    }

    if (grade != 'R') {
        dayAcceptedTotal += acceptedChips;
        dayDefectTotal += defective[i];
    }

    printf("Day %d: Defective = %d, Defect% = %.2f, Accepted = %d, Grade = %c\n",
           i + 1, defective[i], defectPercent, (grade == 'R') ? 0 : acceptedChips, grade);
}

printf("\nOverall Total (processed normal days):\n");
printf("Total Accepted Chips = %d\n", dayAcceptedTotal);

return 0;
}
```

INPUT

```
Day 1 status: 2 defective chips: 50
Day 2 status: 2 defective chips: 120
Day 3 status: 0
Day 4 status: 2 defective chips: 250
Day 5 status: 2 defective chips: 430
Day 6 status: 2 defective chips: 600
Day 7 status: 0
Day 8 status: 2 defective chips: 90
Day 9 status: 2 defective chips: 35
Day 10 status: 1
```

OUTPUT

(no output)

Q7. Arrays

CODE

```
#include <stdio.h>

int main() {
    int n;
    char status[100];
    double used, planLimit, remaining, extraCharge;
    double serverCapacity = 200000.0; // large server total capacity (can be adjusted)
    double serverUsed = 0.0;

    // Input format (example from your test):
    // n
    // status (Active/Inactive)
    // planChoice (1=Basic, 2=Standard, 3=Enterprise)
    // usage(GB)
    // (repeats for n customers)

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf(" %s", status);

        int planChoice;
        scanf("%d", &planChoice);

        scanf("%lf", &used);

        // Inactive accounts → skip processing
        if (status[0] == 'I' || status[0] == 'i') {
            continue;
        }

        // Determine plan limit
        if (planChoice == 1) {
            planLimit = 100.0;
        } else if (planChoice == 2) {
            planLimit = 500.0;
        } else {
            planLimit = 2000.0; // Enterprise 2 TB = 2000 GB
        }

        // If server storage becomes full, stop processing further customers
        if (serverUsed + used > serverCapacity) {
            break;
        }

        serverUsed += used;

        // Remaining storage for this customer's plan
        remaining = planLimit - used;

        // Extra charges only if usage exceeds plan limit
        if (used > planLimit) {
            extraCharge = (used - planLimit) * 4.0; // ₹4 per GB extra
        } else {
            extraCharge = 0.0;
        }

        // Warning if usage exceeds 95% of the plan limit
        double utilization = (used / planLimit) * 100.0;
        if (utilization > 95.0) {
            printf("Customer %d: WARNING - Utilization %.2f%% | Remaining %.2f GB | Extra Charges %.2f\n",
```



```
        i + 1, utilization, remaining, extraCharge);
    } else {
        printf("Customer %d: OK - Utilization %.2f%% | Remaining %.2f GB | Extra Charges %.2f\n",
            i + 1, utilization, remaining, extraCharge);
    }
}

return 0;
}
```

INPUT

```
4
Active
1
120
Inactive
2
600
Active
2
520
Active
3
1900
```

OUTPUT

(no output)

Q8. Arrays

— Not Submitted —

Q9. Arrays

— Not Submitted —

Q10. Arrays

— Not Submitted —

Q11. Arrays

— Not Submitted —

Q12. Arrays

— Not Submitted —

Q13. Arrays

— Not Submitted —

Q14. Arrays

— Not Submitted —

Q15. Arrays

— Not Submitted —

Q16. Arrays

— Not Submitted —

Q17. Arrays

— Not Submitted —

Q18. Arrays

— Not Submitted —

Q19. Arrays

— Not Submitted —

Q20. Arrays

— Not Submitted —